

DEVELOPER SYSTEM DESIGN

Build a Private AI Coding Agent Locally

Setup and design guidelines using Ollama, OpenCode, and custom workspace skills.

SLIDE 2

System Architecture

The Agent Loop Unlike browser chat systems, a coding agent runs an execution loop: reads files, creates code blocks, runs test suites, and diagnoses compile errors.

The Local Link An interface bridge (OpenCode) links global rules, workspace skills, and CLI tools with a local Ollama API engine.

System Dataflow

1. Developer prompts Agent
2. Agent reads rules & skills
3. Local Ollama compiles actions
4. Terminal tools execute files

SLIDE 3

Config file: opencode.json



```
{  
  "provider": "ollama",  
  "endpoint": "http://localhost:11434",  
  "model": "qwen2.5-coder:14b",  
  "default_agent": "builder",  
  "system_instructions_path": "./.agents/instructions.md"  
}
```

SLIDE 4

Defining Workspace Rules

Global Instructions Define style guidelines and safety checks in `.agents/instructions.md` to prevent the model from going rogue.

Examples:

- Use TS strict mode for frontend modules.
- Use async DB sessions for backend.
- Never delete files without developer review.

The Git Sandbox Rule Always commit active changes **before** launching an agent. If the agent makes a mistake, roll back instantly using:

```
$ git reset --hard
```

SLIDE 5

Skills Directory: /skills

Custom Skill Guides Equip the agent with specific framework recipes (e.g. Next.js, FastAPI) to prevent reasoning drift in smaller models.

Location: project-
root/skills/fastapi-api.md

Target FastAPI Guideline

1. Check existing imports in `app/main.py`.
2. Use Pydantic request/response schemas.
3. Include clear HTTPExceptions.
4. Write unit tests adjacent to files.

SLIDE 6

Auditing Security Bounds

Tool Risks Category risk levels dictate the level of execution permissions granted to the agent:

- **Low Risk:** (File Read, Git Diff) Safe to run automatically.
- **Medium Risk:** (Write Files, Run Tests) Execute in Git sandbox.
- **High Risk:** (Delete Files, Terminal Shell) Require step-by-step developer confirmation.

Safety Registry OpenCode prompts for user confirmation before executing any High-Risk action, preventing raw command injection attacks on your host.

SLIDE 7

Executing the Agent Loop

```
≡≡≡ Launching Local Agent Run Simulation ≡≡≡
[Step 1] Loading configs, guidelines, and skills ...
[Step 2] Reading workspace status ...
[Step 3] Querying local qwen2.5-coder:14b ...
    └─ Generation completed in 4.71 seconds.
[Step 4] Extracting tool call payload ...
    └─ Parsed Action: write_file to 'workspace/app/main.py'
[Step 5] Writing modified code ...
[Step 6] Adding health-check assertion to tests ...
[Step 7] Running unittest suite ...
≡≡≡ Workspace Test Results ≡≡≡
Ran 2 tests in 0.013s | OK
[Pass] Agent validation completed successfully!
```


SLIDE 8

Operational Hurdles & Fixes

Tool Permission Fatigue

Automating compiler runs reduces fatigue, but requires sandboxing (Docker dev containers) for secure execution.

Recursive Loop trap Halt execution if the model apologizes and fails tests 3 times consecutively.

Context Window Pruning Prune old file dumps and system summaries from memory history to maintain 16k context accuracy.

Lazy XML Parsers XML regex parsers ignore markdown fences and conversational clutter.